Binary and packages

Two one-file Linux artefacts built from the same frozen binary — a bare executable, a `.deb`, an AppImage and a self-extracting `.run` — plus a Python wheel for a machine that already has Python. All are for a machine without a source checkout.

Build

```
pip install pyinstaller
pip install build            # for the wheel
scripts/build_linux_binary.sh     # -> dist/madrigal-lab
scripts/build_deb.sh              # -> dist/madrigal-protocol-lab_<version>_amd64.deb
scripts/build_appimage.sh         # -> dist/protocol-lab-x86_64.AppImage
scripts/build_run.sh              # -> dist/madrigal-lab.run
python -m build --wheel           # -> dist/madrigal_protocol_lab-<version>-py3-none-any.whl
```

`build_deb.sh`, `build_appimage.sh` and `build_run.sh` package the binary from `build_linux_binary.sh`; run that one first. `build_appimage.sh` downloads `appimagetool` into `.scratch/` and runs it extracted, so no FUSE is needed at build time.

The binary bundles the interpreter, PySide6, the fonts, the docs and the corpus, so it is one file with no install. Build on the oldest glibc you intend to support: the result does not run on an older one.

Size

Artefact	Size
`dist/madrigal-lab`	about 97 MB
`dist/madrigal-protocol-lab_1.2.0_amd64.deb`	about 97 MB
`dist/protocol-lab-x86_64.AppImage`	about 97 MB
`dist/madrigal-lab.run`	about 97 MB
`dist/madrigal_protocol_lab-1.2.0-py3-none-any.whl`	about 175 KB
`dist/madrigal-protocol-lab-docker.tar.gz`	about 451 MB (compressed image)

Most of the size is Qt and the bundled CPython. The binary is not compressed; the AppImage and the `.run` carry the same binary in a SquashFS and a gzipped archive respectively.

Run

```
./dist/madrigal-lab --capture <normalized.json> --rule <rule.json>
```

Without arguments it opens an empty window. On a machine with no display, set `QT_QPA_PLATFORM=offscreen`.

The `.deb` installs the executable at `/usr/bin/madrigal-lab`, the icon and desktop entry under `/usr/share`, and the docs, fonts and corpus under `/usr/share/madrigal-protocol-lab`:

```
sudo dpkg -i dist/madrigal-protocol-lab_1.2.0_amd64.deb
madrigal-lab
```

The AppImage runs from any directory with no install. Where FUSE is absent, extract and run the launcher:

```
chmod +x dist/protocol-lab-x86_64.AppImage
./dist/protocol-lab-x86_64.AppImage --appimage-extract
./squashfs-root/AppRun
```

The `.run` installs the binary, the desktop entry and the icons into `~/.local` (or `$PREFIX`):

```
chmod +x dist/madrigal-lab.run
PREFIX=/tmp/prefix ./dist/madrigal-lab.run
```

The wheel installs the package and its dependencies:

```
pip install dist/madrigal_protocol_lab-1.2.0-py3-none-any.whl
```

Docker

The image carries the source, the tests, the corpus and the pinned dependencies, plus the Qt runtime libraries the window needs. It runs the suite or the window with no host setup:

```
docker build -t madrigal-protocol-lab .
docker run --rm madrigal-protocol-lab pytest tests/ -q
docker run --rm madrigal-protocol-lab python -m src.ui.main_window --help
```

`QT_QPA_PLATFORM=offscreen` is set in the image, so the default `CMD` (`python -m src.ui.main_window`) starts the window headless. A visible window additionally needs the X socket:

```
docker run --rm -e DISPLAY \
    -v /tmp/.X11-unix:/tmp/.X11-unix madrigal-protocol-lab
```

Save and load a built image as one file:

```
docker save madrigal-protocol-lab | gzip > dist/madrigal-protocol-lab-docker.tar.gz
docker load < dist/madrigal-protocol-lab-docker.tar.gz
```

The compressed image is large (about 451 MB): it is Debian slim plus CPython, PySide6 and the Qt libraries, not a reduction of the 97 MB binary, so it is a build-from-`Dockerfile` path rather than a download path.

What was verified

- The binary prints the usage of the window entry point and starts the window headless with a capture and a rule (`QT_QPA_PLATFORM=offscreen`), where it runs until the timeout.
- The `.deb` is built with `dpkg-deb`; extracting it and running `usr/bin/madrigal-lab --help` prints the usage.
- The AppImage is built with `appimagetool`, extracted with `--appimage-extract`, and `squashfs-root/AppRun --help` prints the usage.

- The `.run` is built with `makeself`, installed into a temporary `PREFIX`, and the installed `bin/madrigal-lab --help` prints the usage; the desktop entry and all rendered icon sizes land under the prefix.
- The wheel builds with `python -m build` and reports the version `1.2.0`. A fresh virtualenv installed from the wheel resolves `madrigal-lab` as a console script and prints the window usage.
- The Docker image builds with `docker build`; `docker run --rm ... python -m src.ui.main_window --help` prints the usage and the default `CMD` starts the window headless (offscreen). The Qt runtime libraries are installed in the image, so the window imports without a host `libglib`. The image is exported with `docker save | gzip`.
- The application registers its desktop file id (`madrigal-protocol-lab`) at startup, so a dock resolves the window icon from the installed entry rather than from the title-bar icon alone.
- The package was built and checked on the development host, not installed system-wide, and not run on a second machine.

Not covered

- A real display was not available, so the window was exercised headless only.
- The binary was not tested on Ubuntu 22.04 or another glibc as a packaged artefact; the source suite is checked in those environments (see `docs/PORTABILITY.md`), not this binary.
- The desktop entry assumes `Icon=madrigal-lab` resolves through the hicolor theme; the icon was not checked in a running desktop session.